SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

OracleSort is introducing the strategic solution to the identified problem of students manually matching past year questions to lecture chapters — a fragmented, time-consuming process with no trend insights. OracleSort leverages ILMU-GLM-5.1 as its core reasoning engine to automate chapter detection, question-to-chapter matching, and exam prediction generation.

1.1. *Purpose:*

The system analysis document highlights the technical scope and design-related decisions behind the OracleSort development.
So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
   a. It is a client-side web application built on Next.js with a server-side API proxy. The frontend runs in the browser, while a Next.js API route (/api/proxy) handles GLM API calls server-side to bypass CORS and secure the API key. All persistent data is stored in the browser's localStorage — no backend database is required for the hackathon MVP.
2. ***Data Flows:***
   a. How the data of lecture PDFs and past year papers flows from the user's browser, gets extracted as text by pdf.js client-side, is sent through the /api/proxy route to ILMU-GLM-5.1 for chapter detection and question matching, the structured JSON response is parsed and saved to localStorage, and finally the dashboard components read from localStorage to render trends, predictions, and exportable PDFs.
3. ***Model Process:***
   a. The model shows the end-to-end workflow of the core features: uploading a lecture PDF and detecting chapters, uploading past year papers, triggering the 4-step analysis (Extract → Match → Analyze → Generate), viewing the interactive dashboard, and exporting results as PDFs.
4. ***Role of Reference:***
   a. This document is a reference for all developers and testers to understand the overall high-level architecture of this project, the GLM integration patterns, and the data flow between components.

1.2. ***Background**:*

University students preparing for exams rely on past year questions but have no systematic way to identify which lecture chapters each question belongs to. The process is entirely manual — reading through papers, guessing topic relevance, and hoping their study focus aligns with exam trends.

1. *Previous Version:*
   a. The existing approach is entirely manual: students read lecture slides and past papers side-by-side, highlight topics by hand, and create personal study guides. There

© UMHackathon 2026

is no digital tool that automates this matching process. Some students use generic AI chatbots (ChatGPT, etc.) to ask about topics one question at a time, but this lacks persistence, structured analysis, and trend visibility across multiple papers.

2. *Changes in Major Architectural Components*
   a. *OracleSort introduces a fully automated pipeline: PDF text extraction replaces manual reading, GLM-5.1 AI replaces human judgment for chapter matching, structured localStorage replaces paper-based notes, and an interactive dashboard replaces static study guides. The /api/proxy route is a new component that acts as a secure intermediary between the client and the ILMU API.*
3. *New capabilities that are only introduced in this *
   a. *AI-powered chapter auto-detection from lecture PDFs (with grouping of sub-chapter files), question-to-chapter matching with source document tracking, exam probability predictions with priority tiers, context-aware AI chat assistant, and PDF export of analysis results — none of which exist in any manual study workflow.*

1.3. Target Stakeholder

Stakeholders	**Roles**	**Expectations**
Student	*Uploads lecture PDFs and past year papers, views dashboard predictions, asks AI chat questions, exports PDF study guides*	*Fast chapter detection, accurate question matching, clear priority predictions, study-focused PDF exports*
Educator	*Analyzes question patterns across semesters to adjust lecture emphasis*	*Visual trend data, question frequency per chapter, insight into which topics are over/under-represented*
Developer	*Maintains the codebase, deploys updates, monitors API usage*	*Modular component architecture, clear localStorage abstraction layer, documented API proxy*
Judge (UMHackathon 2026)	*Evaluates the submission against hackathon criteria*	*Mandatory ILMU-GLM-5.1 model usage, working live demo, clear documentation, innovation in AI application*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web Application (Browser-based, responsive)*
Architecture	*Client-Server with Server-side API Proxy*

OracleSort is structured as a Next.js single-page application running entirely in the browser for UI and data storage, with one critical server-side component: the /api/proxy route. The browser client handles all user interactions, PDF text extraction (pdf.js), and localStorage CRUD operations. When AI inference is needed (chapter detection, question matching, predictions, chat), the client sends structured prompts to the /api/proxy Next.js API route, which forwards them to https://api.ilmu.ai/v1/chat/completions with the Authorization: Bearer header and enforces the ilmu-glm-5.1 model. All persistent data (subjects, lectures, papers, chapters, analysis results, chat history) is stored in the browser's localStorage under namespaced keys (oraclesort_subjects, oraclesort_current_subject, oraclesort_chat_{subjectId}).

2.1.2 LLM as Service Layer

The ILMU-GLM-5.1 model is integrated as a service layer through three distinct prompt pipelines, not a generic "AI module":

1. **Chapter Detection Pipeline** — System prompt instructs GLM to detect chapter structure from lecture PDF text. Supports three auto-detected modes: multiple files (groups sub-chapters by filename + content), single file with headings, single file without headings. Returns structured JSON [{number, title, sourceFiles}].
2. **Question Matching Pipeline** — System prompt instructs GLM to extract individual questions from past year paper text and match each to a chapter number. Returns structured JSON [{id, text, type, chapterNumber, sourceDocument}] with optional fields for MCQ options and image descriptions.
3. **Prediction & Chat Pipeline** — System prompt instructs GLM to generate probability predictions per chapter based on question frequency, or to answer student questions using analysis context. Returns structured JSON for predictions, or free-text for chat responses.

2.1.3 Dependency Diagram

a. *How the prompts are being constructed and sent to the GLM API*

© UMHackathon 2026

- *The callAI() function in lib/api.ts constructs prompts as: { max_tokens: 4096, temperature: 0.3, messages: [{ role: "system", content: systemPrompt }, { role: "user", content: userPrompt }] }.*
- *The system prompt is a short, directive instruction (1-2 lines) specifying the exact JSON output format.*
- *The user prompt injects the relevant data: chapter list + question text for matching, question frequency counts for predictions, or analysis context + chat history for the assistant.*
- *This payload is sent to /api/proxy, which adds model: "ilmu-glm-5.1" and the Authorization: Bearer header before forwarding to https://api.ilmu.ai/v1/chat/completions.*

b. *What goes into the context window?*
- *Chapter detection: Lecture PDF text (truncated to 8,000 chars for single, 6,000 for batch) + existing lecture filenames and text excerpts.*
- *Question matching: Chapter list (numbered titles) + past year paper text (truncated to 8,000 chars total across all papers, ~4,000 chars per paper).*
- *Predictions: Chapter question counts + total question count.*
- *Chat: Subject name, chapter frequencies, prediction summaries, analyzed paper names + last 10 chat messages.*

c. *How is your system receiving, parsing the responses and passing to the next system component?*
- *The proxy returns the raw OpenAI-format response: { choices: [{ message: { content: "..." } }] }.*
- *callAI() extracts choices[0].message.content and returns the text string.*
- *Each pipeline function (extractChapters, matchQuestionsToChapters, generatePredictions) strips markdown fences (```json), parses JSON, validates the structure, and maps to typed TypeScript interfaces.*
- *Parsed results are saved to localStorage via updateSubject() and trigger React state updates to re-render the dashboard.*

d. *Where the limitation of token are enforced or input are chunked before it actually reaches to GLm*
- *Lecture PDF text is truncated to 8,000 characters for single-file upload, 6,000 for the new file + 2,000 per existing file in batch uploads.*
- *Past year paper text is truncated to 4,000 characters per paper before being joined and sent for matching (8,000 chars total).*
- *Chat context sends only the last 10 messages to stay within token limits.*
- *max_tokens: 4096 is set on every API call to cap response length.*
- *temperature: 0.3 is used for consistent, structured JSON output.*

© UMHackathon 2026

e. *Also, do outline all the major API calls between the system components [e.g. Customer Mobile App calls "Amazon API Gateway" for Order Service]*
   - *Browser Client → /api/proxy (POST) — All AI inference requests*
   - */api/proxy → https://api.ilmu.ai/v1/chat/completions (POST) — Forwarded GLM API calls*
   - *Browser Client → localStorage (read/write) — All CRUD operations for subjects, chapters, papers, analysis, chat history*
   - *Browser Client → pdf.js (internal) — PDF text extraction from uploaded File objects*

f. *Do show how frontend, backend, services, database and external API interacts with each other [e.g. Order service communicates with Amazon RDS to ensure persistence of order data]*
   - *Frontend (Next.js pages/components) reads/writes localStorage directly via lib/storage.ts abstraction layer.*
   - *Frontend calls /api/proxy (Next.js API route) for all AI operations — the proxy is the only server-side component.*
   - *The proxy communicates with the external ILMU GLM API at https://api.ilmu.ai/v1/chat/completions.*
   - *There is no traditional backend or database — localStorage acts as the persistence layer for the hackathon MVP.*
   - *PDF extraction (pdf.js) and PDF generation (jsPDF) run entirely client-side.*

g. *Must highlights the distributed components if the system is planning to be deployed across multiple services*
   - *The application is deployed as a single Vercel serverless function. The /api/proxy route runs as an Edge/Serverless Function on Vercel. The static frontend is served from Vercel's CDN. There are no multiple distributed services — the architecture is intentionally monolithic for the hackathon timeframe.*

2.1.4. Sequence Diagram

a. ***Upload Lecture PDF and Detect Chapters Flow (Student Journey):***

1. *Student clicks "Lecture Materials" upload card and selects a PDF file.*
2. *handleLectureUpload() creates a lecture entry with status "Processing" and saves to localStorage.*
3. *extractPDFText() (pdf.js) reads the file as ArrayBuffer, iterates all pages, extracts text content.*
4. *extractChapters() constructs a system prompt + user prompt with the extracted text and filename.*
5. *callAI() sends the prompt to /api/proxy.*
6. */api/proxy adds model: "ilmu-glm-5.1" and Authorization: Bearer header, forwards to ILMU API.*
7. *ILMU GLM returns JSON array of chapters.*
8. *callAI() extracts choices[0].message.content, extractChapters() parses the JSON.*

© UMHackathon 2026

9. *Lecture status is updated to "Processed", chapters are saved to the subject in localStorage.*
10. *React state updates, dashboard shows detected chapters.*

b. ***Analyze All Papers Flow (Core Feature Journey):***

1. *Student clicks "Analyze All Papers" button (enabled only when papers exist and chapters are detected).*
2. *handleAnalyzeAll() reads fresh subject data from localStorage.*
3. ***Step 1 (Extracting):** Gathers text from all papers (truncated to 4,000 chars each).*
4. ***Step 2 (Matching):** matchQuestionsToChapters() sends chapter list + paper text to GLM via proxy. GLM returns structured JSON of questions with chapter assignments.*
5. ***Step 3 (Analyzing):** generatePredictions() sends chapter question counts to GLM. GLM returns probability predictions with reasoning.*
6. ***Step 4 (Generating):** Analysis results (matched questions, chapter counts, predictions) are saved to localStorage. Papers are marked "Analyzed".*
7. *Dashboard components read from localStorage and render: stats cards, bar chart, chapter grid, prediction cards, document list.*
8. *Loading overlay animates through all 4 steps with checkmarks and pulsing indicators.*

2.2. Technological Stack

2.2.1. Frontend

Next.js 16 + TypeScript — Server-side rendering with API routes, TypeScript for type-safe component props and API response parsing. Tailwind CSS for utility-first styling with Framer Motion for animations.

2.2.2. Backend

Next.js API Route (/api/proxy) — Lightweight server-side proxy that forwards requests to ILMU GLM API, bypasses browser CORS restrictions, secures the API key server-side (never exposed to the client), and enforces the mandatory ilmu-glm-5.1 model.

2.2.3. Database

localStorage (Browser) — Client-side key-value storage for hackathon MVP. Data stored under namespaced keys (oraclesort_subjects, oraclesort_current_subject, oraclesort_chat_{subjectId}). Abstracted through lib/storage.ts with typed CRUD functions. No server-side persistence required.

2.2.4. Cloud/Deployment

Vercel — Serverless deployment of Next.js application. Frontend served from CDN, API route runs as serverless function. Environment variables (NEXT_PUBLIC_ANTHROPIC_AUTH_TOKEN, NEXT_PUBLIC_ANTHROPIC_BASE_URL, NEXT_PUBLIC_ANTHROPIC_MODEL) configured in Vercel dashboard.

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

© UMHackathon 2026

2.3.1.1. Data Flow Diagram (DFD)

External Entities: Student (browser), ILMU GLM API (external service)

Processes:

- *P1: PDF Text Extraction (pdf.js extracts text from uploaded File objects)*
- *P2: Chapter Detection (GLM analyzes text, returns chapter JSON)*
- *P3: Question Matching (GLM matches questions to chapters)*
- *P4: Prediction Generation (GLM generates probability scores)*
- *P5: PDF Export (jsPDF generates downloadable PDFs)*
- *P6: Chat Assistant (GLM answers context-aware questions)*

Data Stores:

- *DS1: localStorage oraclesort_subjects — Array of Subject objects (each containing lectures, papers, chapters, analysis)*
- *DS2: localStorage oraclesort_current_subject — Current subject ID*
- *DS3: localStorage oraclesort_chat_{id} — Per-subject chat message history*

Data Flows:

- *Student → P1: Upload PDF file*
- *P1 → P2: Extracted text + filename*
- *P2 → ILMU GLM API: Prompt (system + user)*
- *ILMU GLM API → P2: JSON chapter array*
- *P2 → DS1: Save chapters to subject*
- *Student → DS1: Upload past paper (text extracted and stored)*
- *Student → P3: Trigger "Analyze All"*
- *P3 → ILMU GLM API: Chapters + paper text prompt*
- *ILMU GLM API → P3: JSON matched questions array*
- *P3 → P4: Chapter question counts*
- *P4 → ILMU GLM API: Frequency data prompt*
- *ILMU GLM API → P4: JSON predictions object*
- *P4 → DS1: Save analysis results*
- *DS1 → Dashboard: Read and render*
- *Student → P5: Export request*
- *P5 → Student: Downloaded PDF blob*
- *Student → P6: Chat message*
- *P6 → ILMU GLM API: Context + message prompt*
- *ILMU GLM API → P6: Response text*
- *P6 → DS3: Save chat history*

No black holes — all data entering a process either transforms and exits, or is stored. All data stores have both input and output flows.

© UMHackathon 2026

2.3.2. Normalized Database Schema

Since the system uses localStorage (key-value), the logical data model is expressed as TypeScript interfaces:

Subject (root entity, stored as JSON array under oraclesort_subjects):

```
Subject {
  id: string (PK)
  name: string
  lectures: Lecture[]
  papers: Paper[]
  chapters: Chapter[]
  analysis: Analysis | null
}
```

Lecture (dependent on Subject, 1:N):

```
Lecture {
  id: string (PK)
  filename: string
  size: number
  uploadedAt: string (ISO date)
  status: "Processing" | "Processed" | "Failed"
  extractedText: string
}
```

Paper (dependent on Subject, 1:N):

```
Paper {
  id: string (PK)
  filename: string
  size: number
  uploadedAt: string (ISO date)
  status: "Processing" | "Ready" | "Failed" | "Analyzed"
  extractedText: string
}
```

Chapter (dependent on Subject, 1:N):

```
Chapter {
```

© UMHackathon 2026

```
  number: number
  title: string
  sourceFiles: string[] (FK → Lecture.filename)
}
Analysis (dependent on Subject, 1:1):


Analysis {
  matchedQuestions: MatchedQuestion[]
  chapterCounts: Record<number, number>
  predictions: PredictionResult | null
}
MatchedQuestion (dependent on Analysis, 1:N):


MatchedQuestion {
  id: string
  text: string
  type: "mcq" | "short_answer" | "essay" | "image_based"
  options: string[] (optional)
  imageDescription: string (optional)
  chapterNumber: number (FK → Chapter.number)
  sourceDocument: string (FK → Paper.filename)
  marks: number (optional)
}
PredictionResult (dependent on Analysis, 1:1):


PredictionResult {
  predictions: Prediction[]
  summary: string
}
```

The chapterNumber field in MatchedQuestion resolves the N:1 relationship between questions and chapters. The sourceFiles array in Chapter resolves the M:N relationship between chapters and lectures (sub-chapters grouped). The sourceDocument field in MatchedQuestion resolves the N:1 relationship between questions and papers. All relationships are maintained through reference fields rather than junction tables, appropriate for a document-based storage model.

3. *Functional Requirements & Scope*

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

© UMHackathon 2026

3.1. *Minimum Viable Product:*

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026. ***[e.g. in Table]***

#	*Features*	*Description*
1	*Subject Management*	*Students create subject folders (e.g., "Data Structures"), switch between them, and delete them. All data (lectures, papers, analysis) is scoped per subject.*
2	*Lecture PDF Upload + AI Chapter Detection*	*Students upload lecture PDFs (single or batch). pdf.js extracts text, ILMU-GLM-5.1 auto-detects chapters from content and filenames. Supports grouping sub-chapter files (e.g., "1.1.pdf" + "1.2.pdf" → Chapter 1).*
3	*Past Year Paper Upload + Question Matching*	*Students batch-upload past year papers. Text is extracted and stored. "Analyze All Papers" button triggers a 4-step AI pipeline: extract text → match questions to chapters → generate predictions → save results.*
4	*Interactive Dashboard*	*Stats cards (total questions, papers, chapters, top chapter), bar chart of question frequency per chapter (recharts), chapter grid with progress bars, prediction cards with priority tiers (High/Medium/Low), document list table, and slide-in reference panel for per-chapter questions.*
5	*PDF Export*	*Export analysis as separate PDFs (one per chapter) or a single combined PDF. Includes question text, types, source documents, and exam predictions. Generated client-side using jsPDF.*

3.2. *Non-Functional Requirements (NFRs)*

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***

© UMHackathon 2026

Scalability	*OracleSort must handle multiple subjects with multiple PDFs each without browser performance degradation.*	*All data is scoped per subject in localStorage. React state updates are batched. Large PDF text is truncated before processing. Recharts renders efficiently with virtualized DOM.*
Reliability	*If an AI step fails (matching or predictions), partial results must still be saved and displayed. The user should not lose all progress.*	*Each analysis step has independent try/catch. If matching fails, empty results are saved. If predictions fail, results are saved without predictions. Lecture upload failures mark status as "Failed" without blocking other uploads.*
Maintainability	*Each feature (storage, API, PDF, components) is separated into distinct modules with typed interfaces.*	*lib/storage.ts abstracts all localStorage operations. lib/api.ts handles all GLM communication. lib/pdf.ts handl es extraction. Each dashboard component is self-contained with explicit prop types.*
Token Latency	*GLM API calls typically return in 10-30 seconds. Users must see clear progress indication during this time.*	*4-step animated loading overlay with checkmarks for completed steps and pulsing indicator for current step. Each step advances when its API call completes. Chat responses show a typing spinner.*
Cost Efficiency	*Average token cost per analysis session must stay within the 50M token hackathon budget. Prompts should be minimal.*	*System prompts are 1-2 lines (not paragraphs). Lecture text truncated to 8,000 chars. Paper text truncated to 4,000 chars per paper. Chat sends only last 10 messages. Filename-based grouping is attempted before sending content to GLM.*

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026.

© UMHackathon 2026

- *User login and authentication system (currently localStorage-based, no accounts)*
- *Cloud database for cross-device sync (post-hackathon migration to Supabase/Firebase)*
- *Mobile native application (current responsive web app serves mobile browsers)*
- *Collaborative studying features (shared subjects, team analysis)*
- *Answer key auto-grading and score prediction*
- *Scheduled automatic analysis of newly uploaded papers*
- *Multi-language support for non-English lecture materials*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. **Grayscale Rollout & A/B Testing:** *Given the hackathon time constraint, formal grayscale rollout is not feasible. Instead, the team employs a manual testing strategy: each feature is tested in isolation before integration, and the complete analysis flow is tested end-to-end after each major task completion. For post-hackathon, the recommended strategy is to release new features (e.g., cloud sync, answer grading) to 10% of users initially, monitor error rates and API latency, then gradually expand to 100% if no issues are detected.*

4.1.2. **Emergency Rollback (ER) & Golden Release**: *Since the application is deployed on Vercel with Git-based deployments, the Golden Release is the last known stable commit on the main branch. If a deployment introduces critical issues (e.g., proxy route failures, broken dashboard rendering), the team triggers an emergency rollback by re-deploying the previous commit via the Vercel dashboard (vercel rollback). Vercel retains previous deployments for instant rollback. For client-side localStorage data, there is no migration risk since the data schema is forward-compatible (new fields are optional).*

4.1.3. **Priority Matrix:**

Priority	Condition	Action
P1 Critical	ILMU GLM API returns 5xx errors on >50% of requests, or /api/proxy route is down	Display user-facing error message. Do not mark papers as "Failed" — allow retry. Check Vercel function logs. Rollback to last known working deployment if issue is code-side.

© UMHackathon 2026

P2 High	*PDF text extraction fails for uploaded files*	*Mark lecture/paper as "Failed" status. Display error badge. Allow user to delete and re-upload. Log error in Vercel server logs*
P3 Medium	*Dashboard renders incorrectly or charts fail to load*	*Verify data structure in localStorage matches expected TypeScript interfaces. Add defensive null checks. Does not block core analysis flow*
P4 Low	*Chat assistant returns irrelevant or malformed responses*	*Display raw response as fallback. Does not affect analysis or export features. Can be improved with prompt tuning post-hackathon.*

4.2. Monitoring:

For the hackathon MVP, monitoring is implemented through:

1. ***Vercel Server Logs** — The /api/proxy route logs every request (model, message count, payload size) and every response (status code, response length). Errors are logged with full detail for debugging.*

2. ***Client-Side Error Boundaries** — React error boundaries catch rendering failures. Failed API calls throw errors that are caught by individual try/catch blocks, preventing full app crashes.*

3. ***localStorage Health Check** — On app load, getSubjects() safely parses localStorage with try/catch. Corrupt data returns an empty array rather than crashing.*

4. ***Post-Hackathon Plan** — Integrate Vercel Analytics for real-time performance monitoring (Core Web Vitals, API latency percentiles). Add Sentry for client-side error tracking. Set up alerts for P1 conditions (API error rate > 10%).*

4.3. Assumptions: State key operational & environmental conditions that you assumed to be valid while development of system and deployment.

a. Students have stable internet connections on their devices for uploading PDFs and receiving AI responses.

b. Lecture PDFs contain extractable text (not pure scanned images). Image-only PDFs will yield empty text and fail chapter detection.

© UMHackathon 2026

c. The ILMU GLM API (https://api.ilmu.ai/v1/chat/completions) remains available and within the 50M token budget throughout the hackathon period.
d. The ilmu-glm-5.1 model supports the OpenAI-compatible chat completions format and can return structured JSON when instructed.
e. Users access the application via modern browsers (Chrome, Firefox, Edge, Safari) that support localStorage, FileReader API, and ES2020+ features.
f. Uploaded PDFs are under 50MB each (browser memory constraint for ArrayBuffer processing).

External Dependencies: *Do list all the external tools that are applied to your system to function. [e.g. below]*

Tools	*Purpose*	*Risks*
Z Ai GLM API	*Core LLM reasoning engine — handles chapter detection inference, question-to-chapter matching, exam prediction generation, and chat assistant responses*	***HIGH** — If GLM API experiences rate limiting or downtime, core product functionality fails. Mitigation: graceful degradation (partial results saved, retry allowed, user-facing loading states), text truncation to minimize token usage, and fallback empty results rather than crashes.*
pdf.js (pdfjs-dist)	*Client-side PDF text extraction from uploaded lecture and past year paper files*	***LOW** — Well-maintained Mozilla library, runs entirely client-side. Risk: image-only PDFs yield no text. Mitigation: status indicator shows "Failed" for non-extractable PDFs.*
jsPDF	*Client-side PDF generation for exporting analysis results*	***LOW —** Runs entirely client-side, no external dependency. Risk: complex layouts may not render perfectly. Mitigation: simple, structured PDF format with text wrapping and pagination.*

© UMHackathon 2026

Vercel	*Hosting and serverless deployment of Next.js application and API proxy route*	***MEDIUM** — Free tier has function execution time limits (10s for hobby) and bandwidth caps. Mitigation: GLM API calls may exceed 10s; if so, upgrade to Vercel Pro or deploy proxy to a separate server.*
recharts	*Interactive bar chart rendering for question frequency per chapter*	***LOW** — Client-side rendering library. Risk: large datasets may slow rendering. Mitigation: chart data is aggregated (one bar per chapter), always small dataset.*

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

Day	**Phase**	**Deliverables**
Day 1-2	*Setup + Foundation*	*Next.js project extracted from v0, Node.js installed, dependencies resolved, .env.local configured, hydration warning fixed, localStorage CRUD system (lib/storage.ts) implemented*
Day 3-4	*PDF + Chapter Detection*	*pdfjs-dist integrated with dynamic import (SSR-safe), PDF text extraction working, ILMU-GLM-5.1 API proxy route created, chapter detection implemented (auto-detect: multiple files / headings / no headings), batch upload for both lectures and papers*
Day 5-6	*Analysis Pipeline + Dashboard*	*Question-to-chapter matching via GLM, exam prediction generation, 4-step loading overlay, Dashboard*

© UMHackathon 2026

		component (stats cards + recharts bar chart), Predictions (priority tiers), ReferencePanel (slide-in), DocumentList (table), ChapterGrid (cards)
Day 7-8	*Export + Chat*	*jsPDF export modal (separate-by-chapter + combined modes), AI Chat Assistant (floating button, context-aware, localStorage history, suggested questions)*
Day 9-10	*Polish + Documentation + Deploy*	*Console logs removed, error handling hardened, README.md created, Vercel deployment with env vars, SAD documentation, demo video recording*

5.2. Team Member Role:

Member	*Role*	*Responsibilities*
Tan Hock Lai	*Full Stack Developer*	*Next.js frontend development, ILMU-GLM-5.1 API integration (/api/proxy + lib/api.ts), localStorage system, PDF extraction/generation, all React components*
Tan Hock Lai	*Documentation + Testing*	*System Analysis Documentation (SAD), user testing, edge case identification, README and submission documentation*
Tan Hock Lai	*Demo Video + Pitch Deck*	*10 minute demonstration video, pitch presentation slides, feature walkthrough narration, submission package preparation*

5.3. *Recommendations:*

a. **Redis Cache for GLM Responses** — Cache chapter detection and question matching results keyed by PDF content hash. If the same lecture is re-uploaded, return cached results instead of making another API call. This reduces token usage and improves latency.

b. **Supabase Migration** — Replace localStorage with Supabase (PostgreSQL + Auth) for cross-device persistence, user accounts, and collaborative features in the post-hackathon version.

© UMHackathon 2026

c. **Streaming Responses** — Implement Server-Sent Events (SSE) for the chat assistant so users see responses as they're generated, rather than waiting 10-30 seconds for the full response.
d. **Web Worker for PDF Extraction** — Offload pdf.js text extraction to a Web Worker to prevent UI jank when processing large PDFs.
e. **Progressive PDF Chunking** — For PDFs exceeding the context window, implement a two-pass strategy: first pass extracts section headings (cheap), second pass processes only the sections that need detailed analysis.

© UMHackathon 2026